

CMPUF Project Technical Documentation

Stochastic Model, Test Data, Solution Methods, and Results

CMPUF Project

July 17, 2026

Contents

1 Purpose and Software Scope	2
2 Notation and Input Parameters	2
3 Capacity and Scenario Generation	2
4 Implemented Two-Stage Stochastic Model	3
4.1 Variables	3
4.2 Expected-Utility Objective	3
4.3 Constraints	3
5 P-Median Preprocessing	4
6 Solution Algorithms	4
6.1 P-Median Evaluation	4
6.2 Greedy Construction	4
6.3 Local Search	4
6.4 Exact MIP and Scenario Models	4
7 Location Indicators	5
8 Test File Formats	5
8.1 Instance-List File	5
8.2 Standard or Hospital Instance	5
8.3 Geometric Instance	5
8.4 Stochastic Parameter File	5
9 Build and Test Procedure	6
9.1 Implemented Test Plan	6
10 Result File Formats	7
10.1 Naming	7
10.2 Main results2026 File	7
10.3 solgreedy, solopt, and solcent	7
11 Interpretation Caveats	8
12 Code Traceability	8

1 Purpose and Software Scope

The `cmpuf` program solves a capacitated facility location problem in which capacities are subject to random disruptions. Facility locations are first-stage decisions shared by every scenario. Once a capacity scenario is known, customer flows are reoptimized. The implementation is therefore a discrete two-stage stochastic program with continuous recourse.

The maintained executable combines:

- `cpp/cmpuf.cpp`: input, scenarios, indicators, and main program;
- `cpp/model.e.cpp`: CPLEX model, greedy heuristic, and local search;
- `cpp/pmedian.cpp`: p-median preprocessing;
- `cpp/transport.cpp`: independent scenario models;
- `cpp/tri.cpp`: distance-based neighborhood sorting.

2 Notation and Input Parameters

Let $N = \{0, \dots, n-1\}$ be both the customer set and the candidate-location set, $I = \{0, \dots, m-1\}$ the facility set, $R = \{0, \dots, ncap-1\}$ the capacity-state set, and $\Omega = \{0, \dots, K-1\}$ the scenario set, where $K = ncap^m$.

Symbol	Meaning
w_j	Demand of customer j .
$H = \sum_j w_j$	Total demand.
d_{lj}	Distance or cost from location l to customer j , stored as <code>profit[l][j]</code> .
$D_{\max} = \max_{l,j} d_{lj}$	Largest distance in the instance.
C_i^m	Demand assigned to facility i by p-median preprocessing.
γ	Nominal-capacity multiplier.
$S_i = \gamma C_i^m$	Nominal capacity of facility i .
a_r	Probability weight read from <code>walpha</code> .
b_r	Capacity multiplier read from <code>wbeta</code> .
α	Overall disruption parameter.
C_{ki}	Capacity of facility i in scenario k .
P_k	Probability of scenario k .

The member `beta` is read, stored, and printed, but it does not occur in an active capacity formula. Effective multipliers are read directly from `wbeta`.

3 Capacity and Scenario Generation

P-median capacities are scaled as

$$S_i = \gamma C_i^m.$$

The disrupted and nominal states are

$$\text{cap}_{ir} = b_r S_i \quad r = 0, \dots, ncap-2, \quad (1)$$

$$\text{cap}_{i,ncap-1} = S_i. \quad (2)$$

The supplied parameter file therefore produces 0 , $0.33S_i$, $0.66S_i$, and S_i .

Marginal probabilities are

$$p_r = \alpha a_r \quad r = 0, \dots, ncap-2, \quad (3)$$

$$p_{ncap-1} = 1 - \alpha. \quad (4)$$

The a_r values must sum to one. The supplied weights satisfy $0.10 + 0.40 + 0.50 = 1$.

A scenario is $T_k = (T_{k0}, \dots, T_{k,m-1}) \in R^m$. The recursive `Init_T` function enumerates the full Cartesian product and computes

$$C_{ki} = \text{cap}_{i,T_{ki}}, \quad P_k = \prod_{i \in I} p_{T_{ki}}.$$

This produces $K = n \text{cap}^m$ scenarios and approximately $n^2 K$ flow variables. Scenario enumeration is the main scalability limitation.

4 Implemented Two-Stage Stochastic Model

4.1 Variables

First-stage variables are

$$x_{il} = \begin{cases} 1 & \text{if facility } i \text{ is located at node } l, \\ 0 & \text{otherwise.} \end{cases}$$

They are created in $[0, 1]$ and converted to integer variables through `IloConversion`. Recourse variables $z_{ljk} \geq 0$ represent the amount shipped from location l to customer j in scenario k .

4.2 Expected-Utility Objective

The implemented objective is

$$\max \sum_{k \in \Omega} P_k \sum_{l \in N} \sum_{j \in N} (D_{\max} - d_{lj}) z_{ljk}. \quad (5)$$

When total delivered quantity is fixed, this is equivalent to expected transportation-cost minimization. The transformation gives nonnegative coefficients.

Implementation caveat. Demand is imposed as a lower bound rather than an equality. With excess capacity and positive $D_{\max} - d_{lj}$, the solver may ship more than demand. In that case, (5) is not strictly equivalent to minimizing cost for exactly satisfied demand.

4.3 Constraints

Each facility is located exactly once:

$$\sum_{l \in N} x_{il} = 1, \quad i \in I. \quad (6)$$

In the active path, these constraints are added progressively by the greedy heuristic and remain in the model used for the exact solve.

For every location and scenario:

$$\sum_{j \in N} z_{ljk} \leq \sum_{i \in I} C_{ki} x_{il}, \quad l \in N, k \in \Omega. \quad (7)$$

For every customer and scenario:

$$\sum_{l \in N} z_{ljk} \geq w_j, \quad j \in N, k \in \Omega. \quad (8)$$

The optional constraint $\sum_i x_{il} \leq 1$ is implemented by `Init_contraintes_x_x`, but its call is commented out. Facility co-location is therefore allowed.

The x variables are here-and-now decisions, while z variables are scenario-dependent recourse. Benders annotations place x in the master and partition z by scenario. The exact run uses CPLEX's automatic Benders strategy (-1) .

5 P-Median Preprocessing

The preprocessing model uses $X_l \in \{0, \dots, m\}$, the number of medians at node l , and continuous flows z_{lj}^P . It imposes

$$\sum_l X_l = m, \quad (9)$$

$$z_{lj}^P \leq w_j X_l, \quad (10)$$

$$\sum_l z_{lj}^P = w_j. \quad (11)$$

It maximizes $\sum_{l,j} (D_{\max} - d_{lj}) z_{lj}^P$, which is equivalent to distance minimization because demand is satisfied at equality. Since X_l may exceed one, p-median co-location is allowed.

The solution is expanded into m individual facilities. For each one, C_i^m is the flow assigned to its median and becomes its reference capacity in the stochastic model.

6 Solution Algorithms

6.1 P-Median Evaluation

The p-median location vector is fixed in the stochastic model and all scenario flows are optimized. This produces **VMEDIAN1**. A local search from the same vector produces **VMEDIAN2**.

6.2 Greedy Construction

CMpuf::MyGreedyHeuristic locates facilities sequentially:

1. facilities $j > i$ are temporarily fixed to zero;
2. constraint (6) is added for facility i ;
3. CPLEX solves the partial model;
4. the best location of i is recorded and fixed;
5. the temporary constraint is removed.

The final value is **VGREEDY**.

6.3 Local Search

Localsearch performs best-improvement single-facility relocation. All locations are fixed, each facility is released in turn, and CPLEX finds its best new location. The best strictly improving move is accepted until no improvement remains.

If the radius is smaller than n , each facility may move only among the **rayon** closest nodes to its initial position. **ROPTri** sorts these nodes. Local search is run from the greedy solution (**VGREEDYLS**) and from the p-median solution (**VMEDIAN2**).

6.4 Exact MIP and Scenario Models

If argument 2 is **e**, CPLEX solves the full MIP with the requested time limit and relative gap. Internal heuristic frequency and automatic Benders strategy are set to -1 . The code accepts *Optimal*, *OptimalTol*, *SolLim*, and *AbortTimeLim*; consequently, **VOPT** may be an incumbent at the time limit rather than a proven optimum. A best heuristic vector is computed, but its **MIPStart** call is commented out.

After an exact run, **ResolutionScenarios** solves a separate transportation/location model for every known-capacity scenario and fills **SolScenarios** and **SolSupply**. Detailed scenario display is implemented but disabled in the main program.

7 Location Indicators

For $L = (L_0, \dots, L_{m-1})$, centralization is the mean pairwise facility distance:

$$\text{cent}(L) = \frac{\sum_{0 \leq i < j < m} d_{L_i L_j}}{m(m-1)/2}.$$

If $q(L)$ is the number of distinct occupied locations, co-location is

$$\text{coloc}(L) = 100 \frac{m - q(L)}{m}.$$

8 Test File Formats

8.1 Instance-List File

Argument 1 is not an instance itself. It is a text file containing one instance path per line:

```
/absolute/path/to/instance1  
/absolute/path/to/instance2
```

For example, `../BENCHMARK/calgaryhospital15` contains the path of the actual instance. Paths must not contain spaces because they are read with `fscanf(file, "%s", filename)`.

8.2 Standard or Hospital Instance

When `INPUT_TYPE` is not `geometric`, the file contains:

```
n  
w[0] ... w[n-1]  
d[0][0] ... d[0][n-1]  
...  
d[n-1][0] ... d[n-1][n-1]
```

8.3 Geometric Instance

When `INPUT_TYPE` is exactly `geometric`, the reader expects:

```
n  
mtmp  
ptmp[0] ... ptmp[mtmp-1]  
w[0] ... w[n-1]  
d[0][0] ... d[n-1][n-1]
```

The temporary `ptmp` array is read but not subsequently used. Demand and matrix values are extracted through Concert stream operators.

8.4 Stochastic Parameter File

`parameters/param` follows:

```
ncap  
a[0] ... a[ncap-2]  
b[0] ... b[ncap-2]
```

The supplied contents are:

```
4  
0.10 0.40 0.50  
0.00 0.33 0.66
```

9 Build and Test Procedure

From the project root, compile with:

```
make clean
make -j2 cmpuf
```

or:

```
cmake --preset release
cmake --build --preset cmpuf --parallel
```

The executable is `bin/cmpuf`.

The program expects nine user arguments:

```
./bin/cmpuf INSTANCE_LIST MODE M GAP LOCAL_RADIUS \
BB_RADIUS TIME_LIMIT PARAMETER_FILE INPUT_TYPE
```

No.	Argument	Purpose
1	INSTANCE_LIST	File containing one instance path per line.
2	MODE	e enables the exact solution; other values run only heuristic evaluations.
3	M	Number of facilities.
4	GAP	CPLEX relative gap.
5	LOCAL_RADIUS	Local-search neighborhood size.
6	BB_RADIUS	Intended local branch-and-bound radius; calls are currently commented out.
7	TIME_LIMIT	CPLEX limit in seconds.
8	PARAMETER_FILE	File containing $ncap$, a_r , and b_r .
9	INPUT_TYPE	geometric , or conventionally hospital for the standard format.

Example:

```
./bin/cmpuf ../BENCHMARK/calgaryhospital15 e 4 0.00 \
5 40 30000 parameters/param hospital
```

9.1 Implemented Test Plan

For each instance, p-median preprocessing is run once. Then γ takes 1, 1.25, 1.50, and 1.75, while α ranges from 0 to 1 in steps of 0.1. The value of β remains zero. For every pair, the p-median location, greedy solution, and both local searches are evaluated. Mode **e** additionally runs the exact MIP. This gives theoretically $4 \times 11 = 44$ configurations per instance, subject to floating-point loop rounding.

Recommended checks are:

1. run `./bin/cmpuf` without arguments and verify the argument-count error;
2. begin with a small instance and short time limit;
3. verify $\sum_{r < ncap-1} a_r = 1$;
4. ensure $n^2 ncap^m$ fits available memory;
5. verify that `results/hospital` exists and is writable;
6. compare all five objective values.

10 Result File Formats

10.1 Naming

Four hard-coded prefixes are used:

```
results2026_v1_  
solgreedy_v1_  
solopt_v1_  
solcent_v1_
```

The suffix includes part of `argv[1]` followed by `_M_GAP_LOCAL_RADIUS_BB_RADIUS_TIME_LIMIT`.

Current limitation. `concat` uses `argv[1]+16` rather than extracting the final path component. Names depend on the instance-list path length and may be truncated, for example `garyhospital15`. Output directories are hard-coded absolute paths. Every output is opened in append mode.

10.2 Main results2026 File

Each numeric row contains 25 ampersand-separated fields:

No.	Field	Meaning
1	n	Number of nodes.
2	m	Number of facilities.
3–5	Alpha, Beta, Gamma	Test parameters.
6	VOPT	Exact value, or -1 outside exact mode.
7	VGREEDY	Greedy value.
8	VGREEDYLS	Local-search value from greedy.
9	VMEDIAN1	Stochastic value of p-median locations.
10	VMEDIAN2	Local-search value from p-median.
11–15	cent_*	Centralization indicators.
16–20	coloc_*	Co-location percentages.
21	topt	Exact-solution CPLEX time.
22	tgreedy	Greedy time.
23	tgreedyls	Local-search time from greedy.
24	tpmedian1	P-median preprocessing time.
25	tpmedian2	Local-search time from p-median.

Current header anomaly. The header call has 24 `%s` placeholders for 25 labels and no line break. Thus `tpmedian2` is omitted and the first data row is joined to the header. Later numeric rows still contain the documented 25 fields.

10.3 solgreedy, solopt, and solcent

For every (α, γ) pair, `solgreedy` receives:

```
Lls1[0] Lls1[1] ... Lls1[m-1]
```

This is the local-search solution from greedy, not the raw greedy vector.

In exact mode, `solopt` receives:

```
Lopt[0] Lopt[1] ... Lopt[m-1]
```

Also in exact mode, each tab-separated `solcent` row is:

```
alpha    beta    gamma    VOPT    centralizationopt
```

Standard output prints a LaTeX-table-style row for every configuration and can be redirected to a report file.

11 Interpretation Caveats

- The objective is expected utility, and its cost interpretation requires fixed delivered quantities.
- Demand uses $\geq$, so mathematical over-service is possible.
- Facility co-location is allowed.
- `beta` currently has no effect.
- scenarios and variables grow exponentially with m .
- VOPT at the time limit may not be proven optimal.
- repeated runs append to existing output files.
- `while(!feof(file1))` may attempt an additional iteration.

12 Code Traceability

Element	Main function or file
Instance and parameter loading	CMpuf constructor, <code>cpp/cmpuf.cpp</code>
Capacities	CMpuf::Init_cap
Scenarios	CMpuf::Init_T, CMpuf::Init_C
Variables	CMpuf::Init_x, CMpuf::Init_z
Constraints	CMpuf::Init_contraintes, CMpuf::Init_contraintes_x
Objective	CMpuf::Init_obj
P-median	<code>cpp/pmedian.cpp</code>
Greedy heuristic	CMpuf::MyGreedyHeuristic
Local search	CMpuf::Localsearch
Exact solution	CMpuf::Resolution and the main program
Scenario models	CMpuf::ResolutionScenarios, <code>cpp/transport.cpp</code>
Output	AffichageSolutions, AffichageSolutionsScenarios, AffichageCoutTransport